

Hash Table with Separate Chaining

WANG Yizhou

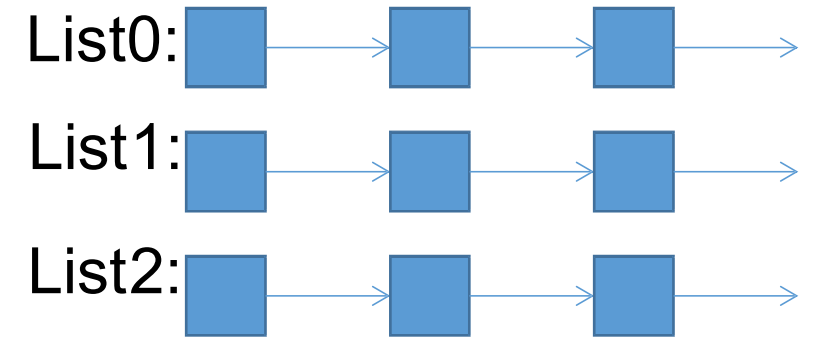
Hash table struct

- The struct used for Hash Table

```
typedef struct node {  
    int key;  
    int value;  
    struct node *next; } List;
```

```
typedef struct hash_sc {  
    int size; // hash table size  
    List ** slots; // array of linked lists } Hash_SC;
```

Slot:



Functions to be implemented

- `List* list_insert(List *L, int key, int value);` // Insert (key, value) to the front of the linked list L
- `list_delete(List *L, int key);` // Delete key from the linked list L
- `int list_find(List *L, int key);` // Find the value based on the key in the L

- `void hash_init(Hash_SC **T, int size);` // Initialize the hash table to the given size
- `void hash_make_empty(Hash_SC *T);` // Make the hash table T empty
- `void hash_insert(Hash_SC *T, int key, int value);` // Insert (key, value) to the hash table T
- `void hash_delete(Hash_SC *T, int key);` // Delete key from the hash table T
- `int hash_contain(Hash_SC *T, int key);` // Check if key exist in the hash table T or not
- `int hash_find(Hash_SC *T, int key);` // Find the value based on the key in the hash table T
- `void hash_free(Hash_SC **T);` // Free the hash table T, if T is not NULL
- `int hash_function(int key, int size);` // The hash function calculates the hash value

Implementation

If key is not in T, then add (key, value) to the front of the list
If key already exists in T, then update the value

```
List* list_insert(List *L, int key, int value){
```

```
    List *tmp;
```

```
    if (list_find(L, key) != INT_MAX) {
```

```
        tmp = L;
```

```
        while (tmp->key != key) tmp  
= tmp->next;
```

```
        tmp->value = value;
```

```
        return L;
```

```
    }
```

Find if the key already exist

```
    tmp = malloc(sizeof(List));
```

```
        tmp->key = key;
```

```
        tmp->value = value;
```

```
        tmp->next = L;
```

```
        L = tmp;
```

```
        return tmp;
```

```
}
```

**create a new pair in
and add to the list**

Implementation

If key does not exist in T, then do nothing
Return the head of the list

```
List* list_delete(List *L, int key){  
    List *tmp, *prev;  
    tmp = L;  
    while (tmp != NULL && tmp->key !=  
key){  
        prev = tmp;  
        tmp = tmp->next;  
    }
```

```
    if (tmp == NULL) return L;
```

key not found

```
    if (tmp == L) {  
        prev = L;  
        tmp = tmp->next;  
        free(prev);  
        return tmp;}  
    prev->next = tmp->next;  
    free(tmp);  
    return L;
```

**node to be
deleted is the
front of the
linked list**

```
    prev->next = tmp->next;  
    free(tmp);  
    return L;
```

**node to be
deleted is not
the front of the
linked list**

```
}
```

Implementation

If key in L, return the value

If key does not exist in L, return INT_MAX

```
int list_find(List *L, int key){  
    List *tmp = L;  
    while (tmp != NULL && tmp->key != key)  
        tmp = tmp->next;  
  
    if (tmp == NULL)  
        return INT_MAX;  
  
    return tmp->value;  
}
```

Implementation

Initialize the hash table to the given size

Create a new empty hash table with the given size

```
void hash_init(Hash_SC ** T, int size){
    Hash_SC *tmpT = malloc(sizeof(Hash_SC));
    tmpT->size = size;
    tmpT->slots = malloc(sizeof(List*)*size);
    int i;
    for (i = 0; i < tmpT->size; i++) {
        tmpT->slots[i] = NULL;
    }
    *T = tmpT;
}
```

Implementation

Make the hash table T empty

All slots are pointing to NULL

```
void hash_make_empty(Hash_SC *T){
    int i;
    for (i = 0; i < T->size; i++) {
        while (T->slots[i] != NULL) {
            T->slots[i] = list_delete(T->slots[i], T->slots[i]->key);
        }
    }
}
```


Implementation

If key is not in T, then add (key, value) to the front of the list

If key already exist in T, then update the value

```
void hash_insert(Hash_SC *T, int key, int value){  
    int hash_value;  
    hash_value = hash_function(key, T->size);  
    T->slots[hash_value] = list_insert(T->slots[hash_value], key, value);  
}
```

Implementation

Delete key from the hash table

If key does not exist in T, then do nothing

```
void hash_delete(Hash_SC *T, int key){  
    int hash_value;  
    hash_value = hash_function(key, T->size);  
    T->slots[hash_value] = list_delete(T->slots[hash_value], key);  
}
```

Implementation

If key in T, return 1

If key not in T, return 0

```
int hash_contain(Hash_SC *T, int key){  
    int hash_value;  
    hash_value = hash_function(key, T->size);  
    if (list_find(T->slots[hash_value], key) == INT_MAX)  
        return 0;  
    return 1;  
}
```

Implementation

If key in T, return the value

If key does not exist in T, return INT_MAX

```
int hash_find(Hash_SC *T, int key){  
    int hash_value;  
    hash_value = hash_function(key, T->size);  
    return list_find(T->slots[hash_value], key);  
}
```

Implementation

Free the hash table T, if T is not NULL

Assign NULL to T

```
void hash_free(Hash_SC **T){
    int i;
    if (*T == NULL) return;
    for (i = 0; i < (*T)->size; i++) {
        while ((*T)->slots[i] != NULL) {
            (*T)->slots[i] = list_delete((*T)->slots[i], (*T)->slots[i]->key);
        }
    }
    free((*T)->slots);
    free(*T);
    *T = NULL;
}
```

Implementation

hash_value = key mod size

key can be negative, but hash_value must be range between 0 and size-1

if key = -1, then hash_value will be size - 1

```
int hash_function(int key, int size) {  
    return ((key % size) + size) % size;  
}
```